

A Reality Check on Learned Traffic Engineering for LEO Mega-Constellations: What Congestion Prices Buy, and Where a Blind One-Pass Baseline Already Suffices

Phuc Hao Do, Nang Hung Van Nguyen, and Minh Tuan Pham

Abstract—Low Earth orbit (LEO) mega-constellations carry traffic over a dense mesh of inter-satellite links whose topology, although time varying, is known in advance from orbital ephemeris. Under load, routing on propagation delay alone concentrates traffic on a few bottleneck links while alternative paths sit idle. The congestion-aware optimum, the user equilibrium, removes this waste but needs the full demand matrix and an iterative solve every slot, and a growing literature amortizes it by learning to predict per-link congestion prices in one forward pass. This paper asks what that buys, and measures it against the baseline the literature has not used: a congestion-blind multipath split costing a single pass, whose path-cost tolerance we sweep rather than fix. With both sides tuned per shell, the learned price field recovers 0.977 of the blind-to-equilibrium gap on the shell it was trained on against 0.668 for the blind split, a paired median difference of 0.317. On three shells it never saw, the paired differences are 0.008, 0.002 and 0.020: the honest reading is parity, not advantage. The same boundary appears on a third axis: under drifting hotspots the proactive variant leads the blind split by 0.159 on the training shell and trails it by 0.247 on an unseen one. Two further results qualify the setting rather than the method. First, the headline gap is usually reported in Bureau of Public Roads travel time, which charges overload in a currency a packet link does not use; under a hard capacity the advantage is an increase in delivered rate of 30.9% rather than a delay reduction of 79.1%, and it survives three loss models (30.9%, 36.5%, 37.1%). Second, the softmax temperature that decodes prices into split flows is treated as a constant of the method and is not one: the setting that maximises recovery moves by one to two orders of magnitude with constellation size, and holding it at the value chosen on the smallest shell costs 0.094, 0.089 and 0.065 per instance elsewhere while costing 0.007 where it was chosen. That loss has been charged to the price field, which did not cause it, so any evaluation of transfer that holds the decoder fixed measures the decoder as much as the model. The inference saving over the iterative solve is real, from 21 to 29 $\times$, but only against a solver started from scratch: warm-started from the previous slot the solver needs 2 to 6 iterations rather than 20, which cuts the saving to roughly 2 to 9 $\times$ and reverses its trend with scale. We also report a negative result: a quantum-walk propagation operator that motivated this study offers no benefit once load features are available, and a plain graph convolution is preferable. Every number is tied to the file, column and aggregation that produce it, and a script recomputes

all of them.

Index Terms—LEO satellite networks, non-terrestrial networks, routing, traffic engineering, graph neural networks, congestion control, user equilibrium, inductive learning, evaluation methodology.

I. INTRODUCTION

LOW Earth orbit (LEO) mega-constellations such as Starlink and Kuiper interconnect their satellites with laser inter-satellite links (ISLs) arranged in a regular grid, forming a wide-area packet network that moves with the constellation. Two features distinguish this network from a terrestrial one. First, the topology is time varying but *deterministic*: the position of every satellite, hence every ISL and its propagation delay, is known in advance from the orbital parameters. Second, the network is large and fast: a single shell contains a thousand or more satellites, and cross-plane links open and close as satellites traverse the constellation.

Routing such a network on propagation delay alone is attractive because the shortest path is cheap and, when lightly loaded, near optimal. Under load it is wasteful: it funnels many flows onto the same minimum-hop links, creating hotspots, while geometrically longer but idle paths are ignored. We measure this waste directly (Section VI-A) and find that, once the network is moderately loaded, congestion-aware routing cuts total travel time by more than half, and by up to a factor of forty under heavy load.

The congestion-aware optimum is classical. Routing every flow so that none can unilaterally lower its cost yields the *user equilibrium* (UE), computed by iterating shortest-path assignment against a load-dependent cost. UE is the right target but expensive: it needs the entire demand matrix and many shortest-path rounds per slot. At constellation scale, with topology changing every few seconds, recomputing UE from scratch each slot is impractical.

This tension, a cheap-but-blind heuristic versus an expensive-but-optimal solve, motivates our approach: *amortize* the equilibrium [38]. A graph neural network (GNN) observes cheap features (per-node traffic intensity and the load that one congestion-blind pass would induce) and predicts, in a single forward pass, the equilibrium congestion price on every ISL. Routing on the predicted cost recovers most of the congestion-aware gain at a fraction of the cost of solving UE.

P. H. Do is with Danang Architecture University, Da Nang, Viet Nam (e-mail: haodp@dau.edu.vn).

N. H. V. Nguyen and M. T. Pham are with the University of Science and Technology, Da Nang, Viet Nam (e-mail: nguyenvan@dut.udn.vn; pm-tuan@dut.udn.vn).

Corresponding author: Minh Tuan Pham.

This research is funded by the Science and Technology Development Fund of The University of Danang under project number B2025-DN02-26.

Because the model's parameters do not depend on the number of satellites, the map is *inductive* in the architectural sense and can be applied unchanged to a larger shell; whether it remains *competitive* there is a separate question and one of the questions we measure. And because the demand pattern drifts predictably (the dense-traffic band rotates under the constellation), feeding the predicted near future demand lets the router act *proactively*, before congestion forms. Figure 1 summarizes the pipeline.

To make the stakes concrete, consider the dense-traffic band passing over a populated longitude. Delay-optimal routing sends the surge of flows along the few minimum-hop ISL chains that cross that region, driving a handful of links far past capacity while the cross-plane links a hop away carry almost nothing. The same traffic spread across the available ISLs would be comfortably feasible, so the loss is one of allocation rather than of capacity. How large that loss looks depends on how the model charges for overload, and we return to this in Section V: a delay-based cost and a loss-based one disagree by more than a factor of two on the same configurations. An operator that recomputes the equilibrium each slot fixes the allocation but pays a heavy, repeated optimization; an operator that routes on stale or blind information does not fix it at all. Amortization offers a third option: spend the optimization once, offline, and let a network predict its outcome thereafter.

This paper asks what that third option actually buys, and the answer turns out to depend sharply on a comparison the literature does not make. Evaluations of learned routing in this setting are anchored between a naive floor, shortest-path or hop-count routing, and the expensive ceiling being amortized. Both are informative and neither is competitive: the floor is weak by construction and the ceiling is the thing whose cost motivated the work. The cell that decides whether the machinery earns its place is the one in between, a policy *as cheap as the learned model* and nonetheless strong. We put one there. Splitting demand across near-shortest paths is congestion-*blind*, costs a single pass, and predates every method discussed here, and a dense +Grid offers it an unusual number of detours to work with.

Measured against that comparator, the learned price field is not uniformly better; it is better in a describable region and not outside it, and saying where the boundary lies is more useful than another aggregate win. A second thread is methodological. Learned traffic engineering is easy to evaluate incorrectly: a greedy baseline that silently drops hard flows, a scaling claim resting on a ratio with a moving denominator, an equilibrium reference measured off its operating point, or a congestion cost borrowed from a domain where its arguments mean something else. We encountered all four, caught them, and distil a protocol (Section V).

Contributions.

- **A comparator this literature has not used.** We evaluate learned congestion prices against blind multipath spreading (an ε -tolerance split) at a matched computational budget. We also record a property specific to satellite topologies: inter-satellite delays take continuous values, so exact cost ties essentially never occur and textbook equal-cost multipath [17] degenerates to single-path routing.

Only a tolerance-based variant is meaningful, and its tolerance is a knob whose setting changes conclusions, so we report the sweep rather than a value.

- **A measured boundary: decisive in distribution, level outside it.** With both sides tuned per shell over fixed grids, the learned field recovers 0.977 of the blind-to-equilibrium gap on the shell it was trained on against 0.668 for blind multipath. (At the fixed decoder setting the same shell reads 0.968 against a strongest cheap rival of 0.869; the tuned comparison above is the fair one.) Paired per unit, the median difference is 0.317. On three shells it never saw, the same comparison reads 0.984 against 0.976, 0.974 against 0.973, and 0.989 against 0.969. It wins by unit count on every shell, but off distribution the margins are small enough that eight units per shell cannot separate them from a tie, and the honest statement is parity rather than advantage.
- **Two separate causes of the off-distribution collapse, of comparable size.** Reported at the manuscript's fixed decoder setting, transfer looks far worse than the above: 0.892, 0.885 and 0.925. Two independent effects account for the difference. First, training on a single shell: holding the instance budget fixed and varying only provenance raises transfer to an unseen shell from 0.898 to 0.976. Second, and previously unremarked, the decoder's soft-min temperature is not a constant of the method. Sweeping it, the setting that maximises recovery moves by one to two orders of magnitude between the 132-satellite shell and the larger ones, and holding it at the fixed value costs a median of 0.065 to 0.094 per unit on every unseen shell. A fixed temperature quietly converts a decoder mismatch into what reads as a failure to generalize.
- **Why blind multipath is a serious baseline here, and where it stops being one.** Tolerance-based spreading recovers 0.668 of the gap on the smallest shell and 0.976 on a shell half again as large, because a larger grid offers more near-shortest detours to spread across. Its strength therefore grows in exactly the regime where a learned map is asked to generalize, which is why it must be in the comparison. What it cannot do is anticipate: it consults path geometry and never load, so it is indifferent to where a hotspot is or where it is going.
- **A caution about borrowed congestion costs.** The Bureau of Public Roads function is a road volume-delay curve in which offered load above capacity is a real state. A packet link with a hard rate cannot occupy that state: the excess is lost, not delayed. Enforcing capacity feasibility shrinks the headline gap from 79.1% measured as delay to 30.9% measured as delivered rate, and the smaller figure is stable across three loss models (30.9%, 36.5%, 37.1%).
- **Honest negative results.** The study began as a quantum-walk GNN; we report the kill-gate record and why a plain graph convolution is the right operator here.

Organization. Section III states the system model and decomposes the amortization problem. Section IV gives the method: the observable features, the edge-price network,

the decoders, and the travel-time bound. Section V is the evaluation protocol, including the baselines, the pitfalls we document, and the setup. Section VI reports the measurements, opening with a table of every headline number and its source. Section VII discusses deployment, threats to validity, what the measurements do and do not license, and future work.

II. RELATED WORK

LEO routing and topology. The deterministic, time-varying structure of LEO ISL networks has been studied for low-latency routing and topology design [1], [2]. Handley showed that a +Grid of ISLs can beat terrestrial fiber on long paths because light is faster in vacuum, making the ISL layer worth routing carefully [1]; Bhattacharjee and Singla explored how the choice of which satellites to interconnect shapes path length and churn [2]. The broader push toward non-terrestrial networks in the 6G era is surveyed in [19], [20], and the design space of competing LEO constellations is compared in [21]; measurement studies of operational systems such as Starlink confirm that capacity, not only geometry, shapes the user experience [22]. Packet-level simulators such as Hypatia [3] and StarPerf [4] reconstruct the moving topology from constellation parameters and have driven work on path stability, handover, and delay under realistic orbital dynamics. This line predominantly routes on delay or hop count and evaluates a lightly loaded network; our contribution is orthogonal and complementary, namely how to route when the bottleneck is congestion rather than geometry, and how to do so without a per-slot optimization.

GNNs for networking. Message passing [23], graph convolutions [5], attention [6], and inductive sampling [7] underpin learning on graphs, whose expressive power [24] and relational inductive biases [25] are now well understood. In networking specifically, GNNs have been used for modeling and control: RouteNet predicts per-path delay and jitter for network modeling and what-if analysis [8]; surveys map the emerging use cases [26]; and learned routing has been pursued by generating distributed protocols [27], by data-driven distance-vector routing [28], and by reinforcement learning with GNNs [9]. These approaches either model performance or learn a control policy by trial and error. Ours is supervised and predicts a congestion-price *field* that is decoded by a single one-shot pass, which keeps inference to one forward evaluation and makes inductivity across constellation size immediate.

Traffic engineering and equilibria. Load-dependent link cost, the user equilibrium, and the system optimum are classical in transportation science [10], [11], [12], [29]: Wardrop's principles characterize the equilibrium, the Beckmann transformation casts it as a convex program solvable by Frank-Wolfe and the method of successive averages [30], [29], and the BPR function is the standard congestion cost. The gap between selfish and optimal routing is the price of anarchy, bounded for such cost classes by Roughgarden and Tardos [13]. In data networks, traffic engineering has a long line of work, from tuning OSPF link weights [31] and responsive-yet-stable load balancing [32] to software-driven wide-area networks [33], [34]. These solve an optimization per measurement epoch; the novelty in LEO is that the topology is

large and changes every few seconds, so a per-slot solve is the bottleneck and amortizing it is the prize.

Learning-based control, and what it is measured against. Deep reinforcement learning has been applied to resource management [35], datacenter traffic optimization [36], and routing over a fixed topology [14], [15]. Such policies require online exploration and are tied to the training topology, and the supervised alternative is to regress onto a computable optimum instead. The closest work to ours takes exactly that route in exactly this setting: *DeepLaDu* [16] trains a graph network to infer per-link congestion prices from the constellation state in a single forward pass for LEO mega-constellations, supervising on Lagrangian duality rather than on a traffic-assignment equilibrium, and jointly optimising ISL connectivity. Its reported comparison is against “non-joint or heuristic baselines” and against the iterative dual solve it amortises.

That comparison structure is the norm rather than the exception, and it is what motivates this paper. Across this line of work the reference points are consistently a naive floor (shortest-path or hop-count routing) and an expensive ceiling (the iterative solve being amortised). What is systematically absent is the cell between them: a baseline that is *as cheap as the learned model* and nonetheless strong. Nor do these evaluations report a regime in which the learned map fails, or test transfer to an unseen constellation *against such a baseline* rather than against the floor. Our contribution is to supply that missing comparator and to map the boundary it reveals.

Multipath spreading as the missing comparator. Splitting traffic over several near-shortest paths is the oldest and cheapest way to relieve congestion without measuring it. Equal-cost multipath [17] hashes flows across next-hops of exactly equal cost and costs one shortest-path computation; responsive load balancing [32] and software-driven WANs [33], [34] generalise the idea with feedback. Spreading is congestion-*blind*: it consults path geometry, never load. That is usually treated as a weakness, and in a terrestrial network with few equal-cost alternatives it is one. A dense LEO +Grid is the opposite case, offering many near-shortest detours between any pair, and we find this changes the comparison qualitatively (Section VI). One caveat is specific to this topology and, to our knowledge, unremarked: with inter-satellite delays taking continuous values, exact cost ties essentially never occur, so textbook ECMP degenerates to single-path routing and only a tolerance-based variant is meaningful.

Quantum walks and learned operators. Continuous-time quantum walks have been used as a graph propagation operator to capture long-range structure [18], and recent function-learning layers such as Kolmogorov-Arnold networks [37] offer alternative inductive biases. We evaluate a quantum-walk operator as a candidate and report it as a negative ablation (Section VI-N).

III. SYSTEM MODEL AND PROBLEM FORMULATION

Table I lists the notation.

TABLE I
NOTATION.

symbol	meaning
$G_t = (V, E_t)$	constellation graph at slot t
prop_{uv}	propagation delay of ISL (u, v)
cap	ISL capacity
$\mathcal{D} = \{(s, d, r)\}$	demand set (source, destination, rate)
x_{uv}	load (aggregate rate) on ISL (u, v)
$\text{cost}_{uv}(x)$	realized link cost under load (BPR)
g_{uv}^*	user-equilibrium congestion price of (u, v)
$\hat{g}_{uv}$	price predicted by the GNN
$h_v^{(l)}$	embedding of node v at layer l
$\text{TTT}(\pi)$	total travel time of routing policy π

A. Time-varying constellation graph

A Walker shell of S satellites in P planes is propagated on circular orbits. Each satellite holds two intra-plane ISLs and up to two cross-plane ISLs, forming a +Grid (Fig. 3). The topology over a horizon is a known sequence $\{G_t\}_t$, $G_t = (V, E_t)$, with $|V| = S$. Each ISL $(u, v) \in E_t$ has a propagation delay $\text{prop}_{uv} = \|p_u - p_v\|/c$ from the known positions p , and a capacity cap . Cross-plane links above a polar cutoff latitude are deactivated, as in operational designs; for a 53° shell the cutoff is never reached, so the ISL graph is near static and the dynamics enter through the demand.

B. Demand, flows, and link load

A demand set $\mathcal{D} = \{(s_k, d_k, r_k)\}$ requests rate r_k from s_k to d_k . We draw demands from a gravity model: a node's attractiveness is $w_v \propto \exp(\kappa \cos(\angle v - \phi_t))$, peaked on a longitude band ϕ_t , and endpoints are sampled with probability proportional to $w_s w_d$. The band ϕ_t rotates between slots, a proxy for the dense-population and sub-solar region rotating under the constellation as Earth turns; the per-slot drift $\Delta = \phi_{t+1} - \phi_t$ is known from the same ephemeris that fixes the topology. This makes the demand non-stationary (the hotspot moves) yet predictable (its motion is known), which is exactly the structure the proactive variant exploits. Offered load is controlled by the number of demands, which we sweep to span the lightly loaded and heavily congested regimes. A routing policy maps each demand k to a path P_k in G_t . Writing r_k for the rate on P_k , the load on an ISL is

$$x_{uv} = \sum_k r_k \mathbb{1}[(u, v) \in P_k], \quad (1)$$

subject to flow conservation at every node $v \notin \{s_k, d_k\}$,

$$\sum_{u:(u,v) \in E_t} f_{uv}^k = \sum_{w:(v,w) \in E_t} f_{vw}^k, \quad (2)$$

with f^k the per-demand flow.

C. Congestion cost and realized travel time

The realized cost of a link under load follows the Bureau of Public Roads (BPR) form [11],

$$\text{cost}_{uv}(x) = \text{prop}_{uv} \left(1 + \alpha (x_{uv}/\text{cap})^\beta\right), \quad \alpha = 0.15, \beta = 4, \quad (3)$$

which is smooth, strictly increasing, and convex in x , and which blows up as a link approaches and exceeds capacity. The total travel time of a policy π inducing loads $x(\pi)$ is

$$\begin{aligned} \text{TTT}(\pi) &= \sum_{(u,v)} x_{uv}(\pi) \text{cost}_{uv}(x_{uv}(\pi)) \\ &= \sum_k r_k \sum_{(u,v) \in P_k} \text{cost}_{uv}(x_{uv}(\pi)). \end{aligned} \quad (4)$$

The two forms are equal because $\sum_k r_k \sum_{(u,v) \in P_k} (\cdot) = \sum_{(u,v)} x_{uv}(\cdot)$; the edge form is the one to evaluate at an equilibrium, a point we return to in Section V.

D. User equilibrium and system optimum

Two operating points bound any router. The *user equilibrium* (UE) satisfies Wardrop's first principle: for every demand, all used paths have equal and minimal cost. Equivalently, UE is the unique minimizer of the Beckmann potential [12]

$$x^{\text{UE}} = \arg \min_{x \in \mathcal{F}} \sum_{(u,v)} \int_0^{x_{uv}} \text{cost}_{uv}(w) dw, \quad (5)$$

over the polytope $\mathcal{F}$ of loads induced by feasible flows (1)–(2); with (3) the integral is $\text{prop}_{uv}(x + \frac{\alpha \text{cap}}{\beta+1} (x/\text{cap})^{\beta+1})$. The *system optimum* (SO) instead minimizes (4) directly, $x^{\text{SO}} = \arg \min_{x \in \mathcal{F}} \text{TTT}(x)$, equivalently routes on the marginal cost $\text{cost}_{uv}(x) + x_{uv} \text{cost}'_{uv}(x)$, and lower-bounds UE. Their ratio $\text{TTT}(\text{UE})/\text{TTT}(\text{SO})$ is the price of anarchy [13]. We use UE as the deployable congestion-aware reference and SO as a tighter ceiling; in our regime they nearly coincide (Section VI).

We compute UE by the method of successive averages (MSA): starting from an all-or-nothing assignment y^0 on free-flow cost, iterate

$$x^{k+1} = x^k + \frac{1}{k+1} (y^k - x^k), \quad y^k = \text{AON}(\text{cost}(x^k)), \quad (6)$$

where AON routes every demand on the shortest path under the current cost; SO uses the same iteration on the marginal cost. Each iteration is $|\mathcal{D}|$ shortest paths and convergence needs T iterations.

E. The amortization problem

Blind shortest path is one AON pass and is congestion blind; UE is T passes and is congestion optimal but does not scale. We seek a parametric map f_θ from cheap, observable state $\mathcal{S}_t$ to a routing π_θ that approaches UE in one forward pass and whose parameters θ are independent of S :

$$\min_{\theta} \mathbb{E}_{\mathcal{D}, G} [\text{TTT}(\pi_\theta)] \quad \text{s.t.} \quad \text{cost}(\pi_\theta) \ll \text{cost}(\text{UE}). \quad (7)$$

Three features of (7) are worth stating plainly, because they decide what a solution can and cannot be. First, the expectation is over *instances*, not over slots of one deployment: θ is chosen once, offline, against a distribution of demand patterns $\mathcal{D}$ and constellation graphs G , and is then held fixed. This is what makes the problem an amortization rather than an online control problem, and it is also where its exposure lies, since nothing forces the deployed instances to be drawn from the distribution θ was fitted to. Section VI measures exactly that exposure.

Second, the constraint is on *inference* cost, not on total cost. Solving the equilibrium once per training instance is permitted and is in fact how the supervision is produced; what must be cheap is the per-slot map at deployment. The two costs are separated by the offline/online boundary, and any honest accounting of the method has to price them separately, which is why Table II reports the per-slot budget in all-or-nothing passes rather than a single wall-clock figure.

Third, and easily missed, π_θ is a *routing*, not a price field. The objective scores the travel time of the flow actually emitted, so any error introduced after the prediction, by the decoder that turns prices into paths, is charged to f_θ as well. A method can therefore fail (7) with a perfectly accurate price field, and we find the decoder is a larger source of loss off distribution than the prediction is.

F. Decomposition into sub-problems

We decompose (7) into four sub-problems (Fig. 2), each mapped to a claim.

The decomposition is not a matter of taste. Problem (7) asks for a map from observable state to a routing that approaches the equilibrium at a fixed inference budget, and that map is a composition of steps which fail for unrelated reasons and are therefore measurable separately. A price field can be predicted accurately and still be decoded into a poor routing, because the equilibrium splits flow while a single shortest path cannot; the decode can be faithful and the prediction still collapse on a constellation the model never saw; and all three can hold at a fixed demand pattern and give way once the pattern moves. Each of these is a distinct failure surface, so each earns its own sub-problem and its own measurement rather than being averaged into one score.

Consistency across the four is enforced structurally rather than assumed. They compose in one direction only: SP2 consumes the field SP1 produces, SP3 asks whether SP1 and SP2 survive a change of constellation, and SP4 asks whether they survive a change of demand. Every sub-problem is scored on the same end-to-end quantity, the realized travel time of the routing that is actually emitted, never on an intermediate such as price-prediction error, so a gain in one stage cannot be claimed unless it survives the stages after it. Section V adds the arithmetic check that the composed pipeline never reports a travel time below the system optimum, which is the bound no routing can cross.

(SP1) Price prediction. Approximate the UE congestion price field

$$g_{uv}^* = \text{cost}_{uv}^{\text{UE}} / \text{prop}_{uv} - 1 \geq 0, \quad \hat{g} = f_\theta(\mathcal{S}_t), \quad (8)$$

so that routing on the cost $\text{prop}(1 + \hat{g})$ approximates the equilibrium flow x^{UE} . Maps to **C1**.

(SP2) Decoding. Turn a price field into a routing. A single shortest-path assignment $\pi = \text{AON}(\text{prop}(1 + \hat{g}))$ is the cheapest decode, but the user equilibrium splits flow across equal-cost paths, so a single path cannot reproduce x^{UE} even with perfect prices (Section IV-E). We therefore use a one-shot *multipath* decode that splits flow downhill on the predicted cost; it costs the same order as the single-path decode and

closes most of that gap. The decode is shared across all demands, so a single $\hat{g}$ routes the whole matrix.

(SP3) Inductive transfer. Require θ to be size invariant so that f_θ trained on $\{G^{\text{small}}\}$ applies to a larger G^{large} zero shot. Maps to **C2**. A message-passing f_θ with shared weights and a local readout satisfies this by construction (Section IV-G).

(SP4) Proactivity. Under drift Δ , predict next-slot demand $\hat{D}_t$ from D_{t-1} and the known Δ , and decode on $f_\theta(\mathcal{S}(\hat{D}_t))$ so the route pre-empts the moving hotspot. Maps to **C4**.

Cost of SP2 plus inference versus the UE solve gives **C3** (Section IV-I).

IV. METHOD

A. Observable features (input to SP1)

For a slot we form per-node features $\xi_v = [\text{out}_v, \text{in}_v, x_v^{0,\text{out}}, x_v^{0,\text{in}}]$, where $\text{out}_v, \text{in}_v$ are the rate originating and terminating at v , and $x_v^{0,\cdot}$ aggregate the *blind load* $x^0 = \text{AON}(\text{prop})$ leaving and entering v . Per-edge features are $[\text{prop}_{uv}, x_{uv}^0]$. The blind load is one AON pass (no iteration) and is the most informative input, since the equilibrium price of a link is largely a function of how overloaded that link would be under naive routing (Section VI-M quantifies this). All features are scaled to be size invariant, so the model does not see absolute coordinates or counts and cannot shortcut on them.

B. Edge-price GNN (model for SP1)

Node embeddings are initialized $h_v^{(0)} = \text{ReLU}(W_{\text{in}} \xi_v)$ and updated for $l = 0, \dots, K-1$ by a symmetric-normalized graph convolution

$$h_v^{(l+1)} = \text{ReLU}\left(W^{(l)} \sum_u \hat{A}_{vu} h_u^{(l)}\right), \quad (9)$$

where $\hat{A} = \tilde{D}^{-1/2}(A+I)\tilde{D}^{-1/2}$ and $\tilde{D}$ is the degree of $A+I$. A per-edge head predicts the *log-price* $m_{uv} = \text{MLP}([h_u, h_v, \text{prop}_{uv}, x_{uv}^0])$, from which the non-negative price is recovered as

$$\hat{g}_{uv} = \max(0, \exp(m_{uv}) - 1). \quad (10)$$

We use $K = 3$ layers and hidden width $d = 32$ throughout; the model has on the order of a few thousand parameters, independent of S .

C. Training objective

We regress the log-price over a dataset $\mathcal{I}$ of (shell, demand) instances:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \frac{1}{|E_i|} \sum_{(u,v) \in E_i} \left(m_{uv} - \log(1 + g_{uv}^*)\right)^2, \quad (11)$$

i.e. the head m_{uv} is regressed directly onto $\log(1 + g_{uv}^*)$, and at inference $\hat{g}_{uv} = \max(0, e^{m_{uv}} - 1)$. The $\log(1 + \cdot)$ scale tames the heavy tail of g^* at saturated links, where the BPR exponent makes the raw price span several orders of magnitude. Targets g^* come from solving UE (6) on the training instances offline; the cost of this offline supervision is paid once and amortized over every deployment slot (Algorithm 1).

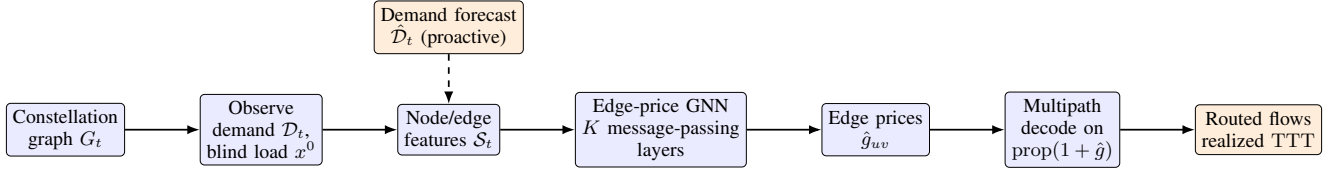


Fig. 1. End-to-end pipeline. Cheap observable state (demand and the load that one congestion-blind pass would induce) feeds an edge-price GNN that predicts the equilibrium congestion price per ISL in one forward pass; a single one-shot multipath decode then routes the whole demand matrix. The proactive variant (dashed) feeds the predicted next-slot demand.

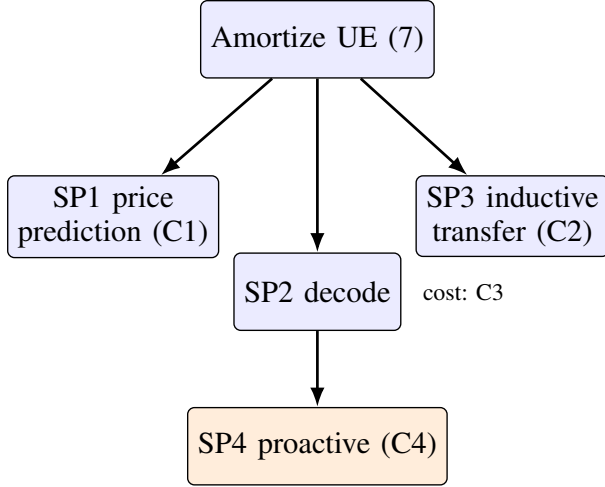


Fig. 2. Decomposition of the amortization problem into four sub-problems, each mapped to a measured claim.

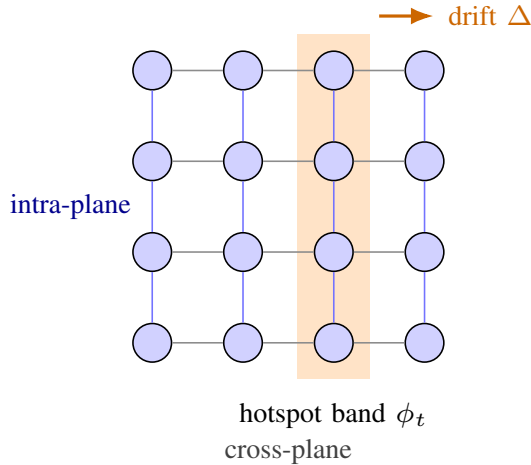


Fig. 3. LEO + Grid ISL topology (a 4×4 patch of the torus; the wraparound links that close the torus are omitted for clarity). Intra-plane links (vertical) are stable; cross-plane links (horizontal) vary with the geometry. The shaded band is the traffic hotspot region, which drifts by a known Δ per slot.

D. Supervision quality and the number of MSA iterations

The quality of the learned price field is bounded by the quality of the targets g^* , so the MSA solve used for supervision must converge well enough that the targets are close to the true equilibrium. We use $T=20$ iterations of (6), after which the average relative change in the load is small and, as a downstream check, the supervised SO and UE respect the

Algorithm 1 Offline supervision

Require: training shells, demand sampler, epochs N

```

1:  $\mathcal{I} \leftarrow \emptyset$ 
2: for each (shell, demand) instance do
3:    $x^0 \leftarrow \text{AON}(\text{prop}); \xi \leftarrow \text{FEATURES}(\mathcal{D}, x^0)$ 
4:    $x^{\text{UE}} \leftarrow \text{MSA}(6); g^* \leftarrow \text{cost}(x^{\text{UE}})/\text{prop} - 1$ 
5:   add  $(\xi, g^*)$  to  $\mathcal{I}$ 
6: end for
7: for  $N$  epochs do
8:   update  $\theta$  by gradient descent on  $\mathcal{L}(\theta)$  (11)
9: end for
10: return  $\theta$ 
  
```

price-of-anarchy ordering of Section V. A larger T tightens the targets at higher offline cost but does not change the deployed model, whose inference is one forward pass regardless; the iteration count is therefore a training-time knob, traded off once and amortized over every slot. Because the offline solve is run on the small training shells, not the large deployment shell, its cost is modest even though MSA itself does not scale, which is the asymmetry the whole method exploits: pay for the equilibrium once, on a small instance, and predict it everywhere else.

E. Decoding: single path and multipath (SP2)

Given the predicted cost $c = \text{prop}(1 + \hat{g})$, the cheapest decode is a single shortest-path assignment $\pi_{\text{sp}} = \text{AON}(c)$. But the user equilibrium splits each demand across all paths of equal cost, so π_{sp} cannot equal x^{UE} even when $\hat{g} = g^*$: the single path carries the whole rate where the equilibrium would have spread it, leaving a *decoding gap* independent of the prediction error. We quantify it in Section VI-D.

We therefore use a one-shot *multipath* decode that recovers the flow splitting at the same asymptotic cost as AON. For each destination d we compute the cost-to-go $\mu_d(\cdot)$ under c with one Dijkstra, then push each demand's rate downhill: at a node u , the rate is split across the distance-decreasing out-edges (u, v) , i.e. $\mu_d(v) < \mu_d(u)$, with the softmax weight

$$w_{uv} \propto \exp(-\text{slack}_{uv}/(\tau \bar{c})), \quad \text{slack}_{uv} = c_{uv} + \mu_d(v) - \mu_d(u) \geq 0, \quad (12)$$

where $\bar{c}$ is the mean edge cost and $\text{slack}_{uv} = 0$ on a shortest path. The downhill edges form a directed acyclic graph (DAG), so the split is loop free and computed in one decreasing- μ_d sweep; $\tau \rightarrow 0$ recovers π_{sp} , larger τ spreads load. The cost is one Dijkstra per destination, the same class as π_{sp} , not

Algorithm 2 One-slot congestion-aware routing

Require: graph G_t , demand $\mathcal{D}$ (or forecast $\hat{\mathcal{D}}$), weights θ

- 1: $x^0 \leftarrow \text{AON}(\text{prop})$ $\triangleright$ one blind pass = features
- 2: $\xi \leftarrow \text{FEATURES}(\mathcal{D}, x^0)$
- 3: $\hat{g} \leftarrow f_\theta(\xi, G_t)$ $\triangleright$ one GNN forward, Eq. (10)
- 4: $\pi \leftarrow \text{MULTIPATHDECODE}(\text{prop}(1+\hat{g}), \mathcal{D})$ $\triangleright$ Eq. (12)
- 5: **return** π

the T -iteration equilibrium solve. Algorithm 2 states one-slot inference with this decode.

F. Travel-time bound (theory)

The decode lets us bound how the prediction error propagates to travel time. Let $\text{TTT}^* = \text{TTT}(\text{UE})$, let π be the multipath decode of $c = \text{prop}(1 + \hat{g})$, and let the realized utilizations stay in a range on which the BPR cost is L -Lipschitz.

Proposition 1: If $\|\hat{g} - g^*\|_\infty \leq \varepsilon$, then

$$\text{TTT}(\pi) \leq (1 + \delta_{\text{dec}} + c_1 H \varepsilon) \text{TTT}^*, \quad (13)$$

where H is the maximum decoded path length (hops), c_1 depends on L and the free-flow delays, and $\delta_{\text{dec}} \geq 0$ is the decoder's gap at exact prices ($\hat{g} = g^*$). The prediction term vanishes as $\varepsilon \rightarrow 0$.

Proof sketch: Per edge, $|c_{uv} - c_{uv}^*| = \text{prop}_{uv} |\hat{g}_{uv} - g_{uv}^*| \leq \text{prop}_{\max} \varepsilon$. Let P_c be a path shortest under the predicted cost c and P^* one shortest under the equilibrium cost c^* , each of at most H hops. Since the per-edge costs differ by at most $\text{prop}_{\max} \varepsilon$, accumulating over at most H hops on each path gives the two-sided additive perturbation

$$\begin{aligned} c^*(P_c) - c^*(P^*) &\leq [c^*(P_c) - c(P_c)] + [c(P_c) - c(P^*)] \\ &\quad + [c(P^*) - c^*(P^*)] \leq 2H \text{prop}_{\max} \varepsilon, \end{aligned}$$

where the first and third bracketed terms are each at most $H \text{prop}_{\max} \varepsilon$ (at most H hops, each edge perturbed by at most $\text{prop}_{\max} \varepsilon$) and the middle term is non-positive because P_c is c -shortest. The multipath split (12) only uses distance-decreasing edges, so this per-demand cost inflation is preserved under aggregation, and on the bounded operating range the induced load displacement is $O(\varepsilon)$. Mapping path cost to realized travel time through the L -Lipschitz BPR response on that range then gives, under the stated bounded-range assumption, a multiplicative factor of order $c_1 H \varepsilon$ with $c_1 = \Theta(L \text{prop}_{\max} / \text{prop})$; here L is finite because the utilizations stay in a fixed range, but it grows in the heavily congested regime, so c_1 is taken on the measured operating range rather than as a universal constant. Setting $\hat{g} = g^*$ ($\varepsilon = 0$) leaves only δ_{dec} , the residual between the softmax split (itself a function of the temperature τ) and the exact equilibrium splitting, which the experiments measure at a few percent. ■

Section VI-D validates (13) empirically: the travel-time gap grows monotonically with the price error and flattens to a small floor δ_{dec} as the error vanishes.

TABLE II

PER-SLOT ROUTING COST IN ALL-OR-NOTHING (AON) SHORTEST-PATH PASSES (T IS THE MSA ITERATION COUNT, ≈ 20). THE GNN ADDS ONE FORWARD TO TWO PASSES; UE/SO NEED T PASSES.

policy	cost (AON passes)
blind, geographic	1
blind multipath (ε -tolerant)	1
one-step correction	2
GNN (ours)	2 + 1 forward
UE, SO (MSA), cold start	$T \approx 20$
UE, SO (MSA), warm start (Section VI-F)	$T \approx 2-6$

G. Inductive transfer (SP3)

The parameters $\theta = \{W_{\text{in}}, W^{(l)}, \text{MLP}\}$ have dimensions set by the feature width and hidden size only, not by S or $|E|$. The aggregation (9) and readout (10) are defined pointwise over nodes and edges. Hence f_θ evaluates on any graph, and a model trained on small shells applies to a larger shell with no change, which is exactly SP3; Section VI-E measures the transfer to a $12\times$ larger shell.

H. Proactive variant (SP4)

Under known drift Δ , the next-slot hotspot band is $\phi_t = \phi_{t-1} + \Delta$, so the demand forecast $\hat{\mathcal{D}}_t$ is available. The proactive router decodes on $f_\theta(S(\hat{\mathcal{D}}_t))$; the reactive baseline decodes on $f_\theta(S(\mathcal{D}_{t-1}))$ and is therefore one step behind the moving hotspot.

I. Complexity (C3)

A forward pass costs $O(K|E|d + K|V|d^2 + |E|d^2)$ for hidden width d ; a decode pass costs $O(|\mathcal{D}|(|E| + |V| \log |V|))$. The full router is one feature pass, one forward, and one decode, i.e. two AON passes plus the forward. UE by MSA (6) costs T AON passes, $T \approx 20$ in our runs when started from scratch. The predicted speedup is thus $\Theta(T)$ up to the forward, and it *grows* with scale because the forward is cheap relative to the repeated routing on a larger graph; the measured factor is $21-22\times$ at the small shells and rises to $29\times$ at 1584 (Section VI-E). Both the prediction and those measurements assume a cold start; Section VI-F measures T for a warm-started solver and finds the picture changes in size and in direction. Table II expresses the per-slot cost of each policy in AON-pass units, the quantity that dominates at scale. Blind and geographic are one pass; one-step is two; the GNN is two passes plus a forward that is asymptotically dominated by a pass; UE and SO are T passes. The GNN thus sits one additive pass above the cheapest heuristics while delivering near-equilibrium quality, and an order of magnitude below the equilibrium solve.

J. Design rationale: why a supervised price field

Three design choices distinguish this approach from the alternatives. First, we predict a *price field* rather than a routing directly: a single $|E|$ -dimensional field is shared by every demand and decoded by an existing, trusted shortest-path router, so the output is always a valid routing and inductivity across

constellation size is immediate. Predicting per-flow paths, by contrast, would scale with the demand matrix and require a learned decoder. Second, we *supervise* on the equilibrium rather than learn by reinforcement: the UE target is computable offline, the objective (11) is a stable regression, and there is no exploration cost or reward-shaping at deployment, which matters when each slot lasts seconds. Third, we predict the equilibrium of a *convex* traffic-assignment problem, so the target is unique and well posed; the network learns to amortize an optimization it could otherwise have to solve from scratch each slot.

V. EVALUATION PROTOCOL

A. Pitfalls that silently distort results

Before reporting results we document four evaluation pitfalls we encountered. Each can silently fabricate or distort a headline number; each has a simple fix. We state them because they are easy to commit in this setting and because our corrected estimator is what produces the numbers in Section VI.

P1: survivorship in greedy baselines. A natural baseline, greedy geographic routing (forward to the neighbor closest to the destination), stalls in local minima on the +Grid ISL graph and fails to deliver 53 to 70% of flows in our loaded regime. Scoring only the delivered flows credits the baseline with a near-optimal mean delay, because the survivors are the short, easy paths. *Fix:* guarantee delivery (here, a shortest-path fallback for stalled flows) and report a survivorship-free total travel time over all demands.

P2: scaling claims on a moving denominator. An early claim that an operator's advantage "grows with the constellation" rested on a relative-percentage gain whose denominator (a baseline error) itself grew super-linearly, so a shrinking percentage masked a widening absolute gap and vice versa. *Fix:* judge scaling on the decision-relevant absolute quantity, not on a ratio with a moving denominator.

P3: measuring an equilibrium off its operating point. The travel time of UE or SO must be evaluated on the converged (MSA-averaged) load using the edge form of (4). Re-routing a single all-or-nothing pass on the converged cost and scoring that instead concentrates flow off the equilibrium split and inflates the reference: in our data it overstated UE travel time by up to $5\times$ (a relative travel time of 0.23 versus the correct 0.05 at high load), which would have made the learned router look closer to optimal than it is. *Fix:* evaluate $TTT = \sum_{(u,v)} x_{uv} \text{cost}_{uv}(x_{uv})$ on the converged load. As a check, the corrected estimator respects $TTT(\text{SO}) \leq TTT(\text{UE})$ at every load, with the price of anarchy growing from 1.00 to 1.12 as load rises; the inflated estimator violated this ordering, which is how we caught it.

P4: a gap-recovery metric saturates when the reference is pathological. The recovered fraction $(TTT_{\text{blind}} - TTT_{\pi}) / (TTT_{\text{blind}} - TTT_{\text{UE}})$ is the standard way this literature reports how close a policy comes to the congestion-aware optimum, and we use it throughout. Its denominator is the blind reference, and on this topology that reference degrades far faster than the equilibrium as the constellation

grows: blind routing costs 2.51 times the equilibrium on the 132-shell, 20.4 times on the 264-shell, and about 770 times on the 1584-shell, because a BPR cost with a fourth power is being evaluated at ever more extreme overload. A denominator that grows by two orders of magnitude makes the fraction insensitive: on the 132-shell our router sits at 1.02 times the equilibrium and scores 0.97; on the 264-shell it sits at 2.64 times and still scores 0.93. Four points of recovered fraction separate a policy that is two percent from equilibrium from one that is more than one hundred and sixty percent from it. Extrapolated to the 1584-shell, a score of 0.94 corresponds to roughly 47 times the equilibrium. *Fix:* report $TTT_{\pi}/TTT_{\text{UE}}$ beside every recovered fraction. It is the quantity an operator experiences, it does not saturate, and it is what makes the three numbers above legible as the very different outcomes they are. We caught this only when an external reviewer asked what the metric compressed, after the fraction had already been the headline of an earlier version of this paper.

None of these four pitfalls is specific to LEO or to graph networks. P1 arises whenever a baseline can decline to act and the metric ignores the declined cases; P2 arises whenever a claim about a trend is read off a normalized quantity with a shifting denominator; P3 arises whenever an iterative solver is scored on a re-derived rather than its converged state; P4 arises whenever a method is scored as a fraction of a gap whose far end is free to run away. The general lesson is to fix a single survivorship-free, absolute, operating-point-correct figure of merit before comparing methods, and to keep a cheap internal consistency check (here, the price-of-anarchy ordering) that a corrupted estimator will fail. We surface these as a reusable evaluation protocol for learned traffic engineering, since each silently produced a publishable-looking but wrong number before it was caught.

B. Setup

We evaluate in the flow-level traffic-assignment framework (BPR cost, user equilibrium), the standard model in which congestion-aware routing is defined and analyzed; this is a deliberate choice that isolates the load-balancing mechanism our method targets, cleanly and reproducibly, before the additional confounds of a packet-level stack (queueing, protocol dynamics) that we leave to a dedicated validation. We simulate Walker shells with a pure-kinematic propagator, fully reproducible from parameters: an Iridium-like shell (66 satellites, diameter 9) and Starlink-like 53° shells of 132 (diameter 16), 264 (28), and 1584 satellites. Capacity is uniform; demands follow the drifting gravity model. We report total travel time (4) relative to blind shortest path (lower is better; blind = 1.00), averaged over multiple demand instances and seeds with standard deviations. Baselines: *blind*; *geographic* greedy (great-circle next hop, with the P1 fallback); *one-step* congestion correction (route on one blind-load pass); *GNN* (ours); and the references *UE* and *SO*. Learned results are param-matched across operators and reported with error bars; all metrics are survivorship free per Section V. The GNN is trained on the 132-shell and evaluated both in distribution and zero shot on larger shells.

TABLE III

EVERY HEADLINE NUMBER IN THIS STUDY AND WHERE IT IS DERIVED. RECOVERED FRACTION IS $(\text{blind} - \text{policy})/(\text{blind} - \text{UE})$; THE UNIT OF ANALYSIS IS (SHELL, SEED, SLOT) THROUGHOUT, AND EVERY VALUE IS THE MEDIAN OVER THOSE UNITS. BLOCKS COME FROM DIFFERENT EXPERIMENTS AND ARE LABELLED WITH THEIR RUN SET: ENTRIES FOR THE SAME SHELL AND THE SAME τ THEREFORE DIFFER BETWEEN BLOCKS, AND SECTION VI-K POOLS THEM RATHER THAN PICKING ONE. READ DOWN A BLOCK, NOT ACROSS BLOCKS.

question	quantity	value
<i>Does the headline survive capacity feasibility?</i>		
gap under BPR delay, offered load $> 4\times$		79.1%
gap in delivered rate, same rows		30.9%
bottleneck loss instead of compounding		36.5%
max-min fair sharing		37.1%
<i>Does it beat a blind baseline at matched budget?</i>		
<i>both sides tuned per shell (Table X); run set r2_7_fair_tuned_wide</i>		
learned / blind, training shell		0.977 / 0.668
learned / blind, unseen 264		0.974 / 0.973
<i>at the fixed decoder setting $\tau = 0.2$; same run set</i>		
learned / blind, unseen 264		0.885 / 0.973
<i>How far does it transfer, at the fixed $\tau = 0.2$?</i>		
<i>a DIFFERENT run set (r2_7_eps_sweep); see Section VI-K</i>		
GNN / blind multipath, 132 sat (trained)		0.975 / 0.660
GNN / blind multipath, 198 sat		0.913 / 0.968
GNN / blind multipath, 264 sat		0.915 / 0.970
GNN / blind multipath, 396 sat		0.947 / 0.960
GNN / blind multipath, 264 sat at 70°		0.941 / 0.944
<i>Does diverse training restore it? (same instance budget)</i>		
single-shell training, unseen 264		0.898
mixed training, unseen 264		0.976
<i>Does proactivity reach an axis blind multipath cannot?</i>		
proactive / blind multipath, trained shell		0.961 / 0.801
proactive / blind multipath, unseen shell		0.737 / 0.984

VI. RESULTS

Table III collects every headline number in this study with the file it is derived from; the subsections that follow derive them. Throughout, *recovered fraction* is $(\text{blind} - \text{policy})/(\text{blind} - \text{UE})$, the share of the blind-to-equilibrium gap a policy closes, and the unit of analysis is (shell, seed, slot).

A. Headroom for congestion awareness

Figure 4 sweeps offered load. At light load, blind shortest path is near optimal and UE offers nothing. As load grows, blind concentrates traffic on bottleneck links (its peak utilization reaches up to nine times capacity while alternates idle) and the gain of UE over blind rises to 9, 55, 90, and 98%. The congestion task, not static routing, is where a learned router helps.

Table IV gives the absolute numbers behind the figure for the 132-shell. The ratio $\text{TTT}(\text{blind})/\text{TTT}(\text{UE})$ widens from 1.0 at light load to about $41\times$ at the heaviest, because the BPR exponent $\beta=4$ makes the cost on an overloaded link grow as the fourth power of its utilization, which blind routing drives to nine times capacity on the bottleneck while leaving parallel links idle. Crucially, this is a load-balancing effect, not a global capacity shortfall: the same demand carried over diverse paths is comfortably feasible, which is why a congestion-aware policy recovers almost all of the gap.

The corrected estimator of Section V also lets us read off the price of anarchy. Solving SO by the marginal-cost iteration and

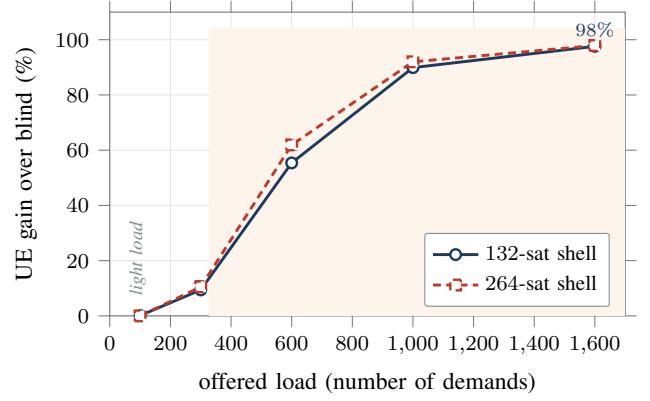


Fig. 4. Congestion headroom: the gain of the user equilibrium over congestion-blind shortest path grows with load, to 98% under heavy load, and is negligible at light load. The shaded region marks the congested regime where a learned router earns its place.

TABLE IV
ABSOLUTE TRAVEL TIME (ARBITRARY UNITS) OF BLIND VERSUS UE ON THE 132-SHELL AS OFFERED LOAD GROWS, AND THEIR RATIO. MEAN OVER SEEDS.

demands	100	300	600	1000	1600
blind	4.8	17.2	78.6	796.6	7433.5
UE	4.8	15.5	35.0	79.7	180.6
ratio	1.0	1.1	2.2	10.0	41.2

comparing to UE, Table V shows $\text{TTT}(\text{SO}) \leq \text{TTT}(\text{UE})$ at every load, with the ratio rising from 1.00 at light load to 1.12 under heavy load. The two operating points nearly coincide, so the learned router's target (UE) is within a few percent of the true system optimum, and matching UE is matching the optimum for practical purposes.

B. The GNN approaches UE and beats every cheap baseline (C1)

Figure 5 and Table VI report TTT relative to blind (the GNN uses the multipath decode of Section VI-D; all values are mean $\pm$ std over seeds and instances). In distribution, the GNN (0.42) closely approaches UE (0.40), recovering about 0.97 of the blind-to-UE gain; SO (0.39) and UE nearly coincide, so the target is essentially the system optimum. The cheap alternatives fail: one-step correction overshoots and oscillates (relative travel time 1.04, *worse* than blind, because rerouting onto the apparently idle alternates creates fresh hotspots), and geographic greedy stalls on the ISL grid, needing the fallback for 53 to 70% of flows (53% at the reported operating point) and still reaching only 0.78. The contrast between the GNN and one-step is the central evidence that the problem is not trivial: a single congestion-blind pass tells you where the hotspots are, but reacting to it by rerouting everyone onto the apparent alternates simply relocates the hotspots, a one-step oscillation that overshoots the equilibrium. The GNN, trained on the converged prices, predicts the *fixed point* of that process directly and so avoids the overshoot. This is why a learned price field, rather than a hand-tuned correction, is needed.

TABLE V

PRICE OF ANARCHY $\text{TTT}(\text{UE})/\text{TTT}(\text{SO})$ ON THE 132-SHELL, MEAN $\pm$ STD OVER $n=3$ SEEDS; ≥ 1 AT EVERY LOAD CONFIRMS THE CORRECTED ESTIMATOR (SECTION V). THE SPREAD IS SMALL BUT NOT ZERO: AT TWO DECIMALS IT PRINTED AS $\pm.00$, WHICH READS AS EXACT.

demands	300	600	1000
$\text{TTT}(\text{UE})/\text{TTT}(\text{SO})$	$1.003 \pm .001$	$1.030 \pm .004$	$1.117 \pm .008$

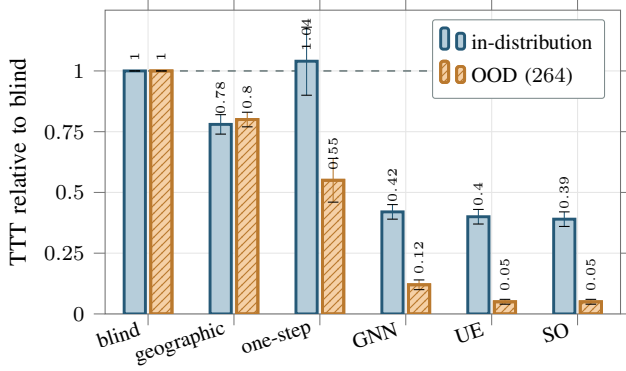


Fig. 5. Travel time relative to blind (lower is better; the dashed line is the blind reference at 1.0). The GNN approaches UE in distribution and is the best deployable policy among the comparators shown here; the cheap heuristics fail (one-step is worse than blind in distribution, geographic stalls). UE and SO nearly coincide.

C. Comparison to a learned router

The natural learned alternative to predicting a price *field* is to predict the *routing* directly. We implement GNN-NextHop, a graph network with the same observable features (demand and blind load) plus a destination indicator, trained to regress the equilibrium cost-to-go to each destination and decoded by greedy next-hop descent (with a shortest-path fallback for stalled flows, so travel time stays comparable). It is a fair, non-trivial learned baseline, and it loses on both axes that matter. In quality, its relative travel time is $0.74 \pm .06$ in distribution and $0.92 \pm .06$ zero shot, versus 0.42 and 0.12 for our edge-price router, because greedy descent on a regressed potential stalls in false local minima: its greedy delivery rate is only 51% in distribution and 21% zero shot (the rest fall back to shortest path), and the potential transfers poorly to the larger shell. In cost, predicting a per-destination potential needs one forward pass *per distinct destination*, not per demand pair: our gravity demand sets concentrate on 91 to 196 distinct destinations per slot across the shells reported here, whereas the edge-price field is predicted once and routes the whole demand matrix in a single forward. Predicting the price and decoding with a classical router is thus both more accurate and far cheaper than learning the routing end to end.

D. Single-path versus multipath decoding, and the bound

Table VII isolates the decoder. The single-path decode of the predicted prices leaves a large gap, especially out of distribution, because it cannot reproduce the multi-path equilibrium even when the prices are good; the one-shot multipath decode (12) closes most of it, raising the recovered fraction from 0.90

TABLE VI

TOTAL TRAVEL TIME RELATIVE TO BLIND (LOWER IS BETTER; BLIND = 1.00). MEAN $\pm$ STD OVER $n=9$ INSTANCE-SEED UNITS PER SPLIT. SO/UE ARE REFERENCES; THE GNN IS DEPLOYABLE, AND SO IS THE BLIND MULTIPATH BASELINE OF TABLE X, WHICH THIS TABLE PREDATES AND DOES NOT INCLUDE.

split	SO	UE	geographic	one-step	GNN
in-distribution	$0.39 \pm .03$	$0.40 \pm .03$	$0.78 \pm .04$	$1.04 \pm .14$	$0.42 \pm .03$
OOD (264)	$0.05 \pm .01$	$0.05 \pm .01$	$0.80 \pm .03$	$0.55 \pm .09$	$0.12 \pm .02$

TABLE VII

DECODER COMPARISON: TTT RELATIVE TO BLIND AND RECOVERED FRACTION OF THE BLIND-TO-UE GAIN, MEAN $\pm$ STD OVER $n=9$ UNITS PER SPLIT. THE MULTIPATH DECODE (12) CLOSES MOST OF THE SINGLE-PATH DECODING GAP. *Run set*: THIS TABLE IS COMPUTED FROM ITS OWN INDEPENDENT SET OF SEEDS. POOLING EVERY UNIT WE HAVE MEASURED FOR THE 264-SHELL AT $\tau = 0.2$, ACROSS ALL THREE RUN SETS, GIVES A MEDIAN OF 0.889 SPANNING 0.838 TO 0.923; THE ENTRIES HERE AND IN THE OTHER TABLES ARE POINTS INSIDE THAT RANGE, NOT CORRECTIONS OF ONE ANOTHER (SECTION VI-K).

	single-path	multipath	UE
in-dist (TTT/blind)	$0.47 \pm .03$	$0.42 \pm .03$	0.40
recovered	$0.90 \pm .05$	$0.97 \pm .01$	—
OOD (264, TTT/blind)	$0.39 \pm .13$	$0.11 \pm .01$	0.05
recovered	$0.64 \pm .14$	$0.94 \pm .01$	—

to 0.97 in distribution and from 0.64 to 0.94 zero shot on the 264-shell, at the same order of cost (all entries are mean $\pm$ std over seeds and instances). This decoder-comparison run is independent of the headline tables and agrees with them within one standard deviation (the 264 multipath GNN reads 0.11 here and 0.12 in Table VI). The multipath decode is the deployed method and is what Table VI and the scaling results report; the single-path decode here serves to expose the decoding gap it closes.

Figure 6 validates Proposition 1. Starting from the true equilibrium prices and injecting growing noise, the travel-time gap to UE rises monotonically with the price error and, as the error vanishes, flattens to a small floor: 3.6% for the multipath decode (the residual δ_{dec} of (13)) versus 12.7% for the single-path decode. The multipath decode is both lower and far less sensitive to prediction error, matching the bound.

E. Inductive transfer to a $12\times$ larger shell, and cost (C2, C3)

Trained only on the 132-satellite shell, the model is applied zero shot to the 264- and 1584-satellite shells. Table VIII reports the recovered fraction of the blind-to-UE gain and the inference speedup over the UE solve. The GNN recovers 0.93 at 264 and $0.94 \pm .02$ at 1584 (a $12\times$ size jump), remaining the best deployable policy throughout, while the speedup over MSA is $21\text{--}22\times$ at the small shells and *rises* to $29\times$ at 1584 (Fig. 7). Against a cold-started solver the amortization therefore appears to pay off more, not less, at the mega-constellation scale that matters operationally. Section VI-F shows that this reading does not survive the comparator an operator would actually run. The transfer is genuinely zero shot: the 1584-shell is never seen during training, has a different number of planes and satellites per plane, and a larger diameter, yet the same weights produce a near-equilibrium

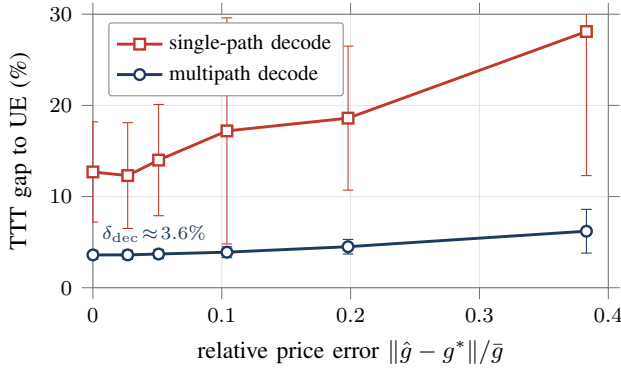


Fig. 6. Empirical validation of Proposition 1: the travel-time gap to UE grows monotonically with the price error and flattens to a small decoder floor as the error vanishes. The multipath decode is both lower and far less error-sensitive.

TABLE VIII

INDUCTIVE TRANSFER (TRAIN ON 132) AND INFERENCE COST. RECOVERED FRACTION OF THE BLIND-TO-UE GAIN (MEAN $\pm$ STD), AND ONE-SHOT GNN ROUTING VERSUS THE MSA SOLVE. ALL ROWS USE THE FIXED DECODER TEMPERATURE $\tau = 0.2$; SECTION VI-J SHOWS THIS SETTING IS NEAR-OPTIMAL ONLY ON THE TRAINING SHELL AND DEPRESSES EVERY TRANSFER NUMBER HERE. *Run set*: THIS TABLE IS COMPUTED FROM ITS OWN INDEPENDENT SET OF SEEDS. POOLING EVERY UNIT WE HAVE MEASURED FOR THE 264-SHELL AT $\tau = 0.2$, ACROSS ALL THREE RUN SETS, GIVES A MEDIAN OF 0.889 SPANNING 0.838 TO 0.923; THE ENTRIES HERE AND IN THE OTHER TABLES ARE POINTS INSIDE THAT RANGE, NOT CORRECTIONS OF ONE ANOTHER (SECTION VI-K).

shell	sats	recovered	GNN (s)	speedup vs UE
in-dist	132	0.97 $\pm$.02	0.05	22 $\times$
OOD	264	0.93 $\pm$.02	0.23	21 $\times$
OOD	1584	0.94 $\pm$.02	8.4	29 $\times$

price field on it. This works because the learned map is from local load structure to a local price, a relation that is scale invariant in a way the absolute routing is not, and because the multipath decode realizes the predicted equilibrium faithfully; the recovered fraction stays near 0.94 and its variance stays low (± 0.02) all the way to the largest shell. The 264-shell recovered fraction reads 0.93 here (Table VIII) and 0.94 in the decoder and operator tables (Tables VII, XI); the one-point spread is the same independent-run variation as the 0.11-versus-0.12 travel time noted in Section VI-D, not a discrepancy. We report the 1584 result as a recovered fraction rather than a travel-time ratio because at that scale blind travel time is far off the range that frames the smaller shells, so the fraction of the blind-to-UE gain is the meaningful, scale-free figure.

F. What the speedup is measured against

The factors above compare a one-shot forward pass with an MSA solve started from scratch. That is the right comparison only if an operator would in fact start from scratch each slot, and in this setting they would not. The constellation is deterministic and known in advance, consecutive slots differ by a small rotation, and satellite identity is stable, so last slot's equilibrium load indexes the same links and is a natural starting point. A warm-started solver is therefore the

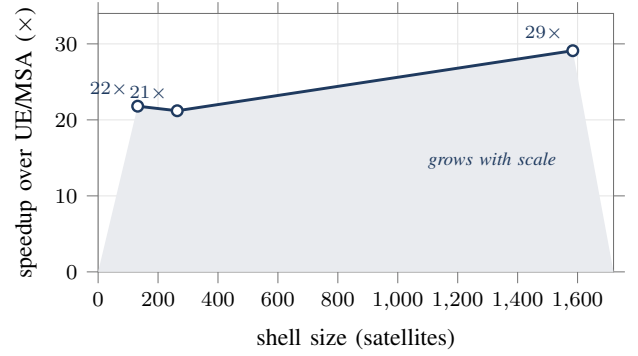


Fig. 7. Inference speedup of the one-shot GNN router over the MSA equilibrium solve, versus shell size. The advantage grows with the constellation, reaching 29 $\times$ at 1584 satellites.

comparator the deployment actually offers, and it is a harder one.

We measure it directly. For each slot we take a long MSA run as the reference and count the iterations a solver needs to come within 0.5% of it, once from the all-or-nothing pass and once from the previous slot's converged load, resuming the step sequence at the iteration the cache was left at rather than restarting it. Cold, the median is 14 iterations on the 132-shell and 22 on the 264-shell, which is where the $T \approx 20$ of Table II comes from. Warm, the medians are 6 and 2. The warm start is cheaper on every one of the 18 units measured.

Two things follow, and the second is the more uncomfortable. First, the speedup shrinks by the ratio of those counts: the 22 $\times$ we report on the 132-shell becomes roughly 9 $\times$ against a warm-started solver, and the 21 $\times$ on the 264-shell becomes roughly 2 $\times$. The saving is real but it is not the order of magnitude the cold-start comparison suggests. Second, the warm start needs *fewer* iterations on the larger shell, not more, so the learned router's margin over it narrows as the constellation grows. That is the opposite direction from the one we claimed: measured against a cold start the amortization looks like it pays off more at scale, and measured against the comparator an operator would actually run, it pays off less. With three seeds and three slots per shell we can state the direction, which is consistent across all 18 units, but not the magnitude.

We leave the cold-start figures in Table VIII because they are what the literature reports and they make our numbers comparable to it, but the operationally honest reading of the cost contribution is the warm-start one, and it is weaker.

G. Proactive routing under drift (C4)

Figure 8 sweeps the per-slot hotspot drift. At zero drift the reactive and proactive routers are identical, as they must be. As the hotspot moves, the reactive lag-1 router routes for yesterday's load and degrades, passing blind at moderate drift and reaching 1.6 $\times$ blind at the largest drift. The proactive router, fed the predicted demand, tracks the clairvoyant equilibrium throughout; the proactive advantage grows from 0 to 76%. The zero-drift control is important: it confirms that the proactive gain is genuinely due to anticipating the hotspot and

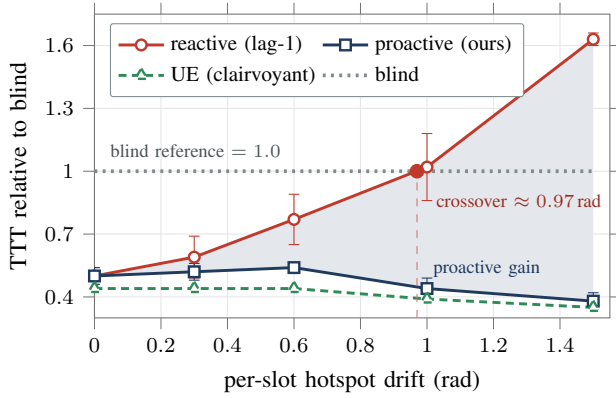


Fig. 8. Proactive versus reactive routing under drifting hotspots (shaded band is the proactive advantage). The reactive lag-1 router degrades past the blind reference once the drift exceeds about one radian; the proactive router tracks the clairvoyant optimum throughout.

not to any artifact of the two pipelines, which are identical when there is nothing to anticipate. The crossover where the reactive router passes blind (around a per-slot drift of one radian in our sweep) marks the operating point beyond which a stale congestion estimate is worse than ignoring congestion altogether, a concrete criterion for when proactivity stops being optional. Because the GNN's input is just a demand-derived feature, switching from reactive to proactive requires no change to the model, only feeding it the forecast instead of the last slot.

That is the whole of the story on the shell the model was trained on. Off it, the story is the one the previous subsections have already told twice. Placing the same blind multipath baseline against the proactive router, with its tolerance tuned per shell and the comparison paired per unit, the proactive router is ahead by 0.166 at zero drift and 0.159 at the largest drift on the training shell, winning every unit; on the unseen 264-satellite shell it is *behind* by 0.209 and 0.247 at those same two settings, and it loses every unit at every drift level we swept. The zero-drift column is the informative one: the deficit is already there when there is nothing to anticipate, so it is not a failure of the forecast but the same effect seen in Section VI-I, namely that blind spreading has more room to work on a larger grid. Proactivity buys what we claimed it buys, and it buys it only where the price field was learned.

This is the third axis to give the same answer. The steady-load comparison, the transfer comparison and the drift comparison were designed to probe different things, and all three place the boundary in the same place: decisive on the training shell, level or behind a one-pass blind policy off it. We take the consistency as the finding rather than as three separate disappointments.

H. Does the headline survive capacity feasibility?

The gap reported above is measured in BPR travel time, and Section V argues that this charges overload in a currency the link does not use. We therefore recompute it under a hard capacity: a link offered L with capacity C carries $\min(L, C)$, each flow crossing it keeps $\min(1, C/L)$, and end-to-end

TABLE IX
THE CONGESTION-AWARE ADVANTAGE MEASURED TWO WAYS ON THE SAME CONFIGURATIONS. THE DELAY COLUMN IS THE BPR TRAVEL-TIME REDUCTION; THE THREE LOSS COLUMNS ARE THE INCREASE IN DELIVERED RATE UNDER THREE FEASIBILITY MODELS THAT DISAGREE ABOUT HOW MULTI-HOP LOSS COMPOUNDS. ROWS ARE BANDS OF THE OFFERED LOAD BLIND ROUTING PLACES ON ITS BUSIEST LINK; MEDIAN OVER (SHELL, LOAD, SEED).

offered load	n	delay (%)	delivered rate (%)		
			(BPR)	product	bottleneck
$\leq 1\times$	6	0.0	0.0	0.0	0.0
$1-2\times$	3	6.2	29.9	11.8	12.3
$2-4\times$	9	45.6	35.4	22.8	21.6
$> 4\times$	12	79.1	30.9	36.5	37.1

TABLE X
LEARNED PRICE FIELD AGAINST BLIND MULTIPATH SPREADING, BOTH TUNED PER SHELL OVER FIXED GRIDS, EIGHT INSTANCES EACH. RECOVERED FRACTION OF THE BLIND-TO-EQUILIBRIUM GAP. THE GAP COLUMN IS THE MEDIAN OF THE PER-INSTANCE DIFFERENCE, NOT THE DIFFERENCE OF THE MEDIANS. τ IS THE DECODER TEMPERATURE AND ε THE PATH-COST TOLERANCE.

shell	learned			blind		gap	wins
	$\tau=0.2$	best	τ^*	best	ε^*		
132, 53° (trained)	0.968	0.977	0.1	0.668	0.1	+0.317	8/8
198, 53°	0.892	0.984	3.5	0.976	0.2	+0.008	8/8
264, 53°	0.885	0.974	8	0.973	0.2	+0.002	6/8
264, 70°	0.925	0.989	2	0.969	0.5	+0.020	8/8

delivery is the product along the path. The configurations, shells, loads and seeds are those of Section VI-A, so the rows correspond one to one.

Table IX reports the result. On configurations where blind routing offers a link more than four times its capacity, the congestion-aware advantage reads 79.1% as a reduction in travel time and 30.9% as an increase in delivered rate. Both are real; they measure different things, and only the second is available to a link that drops what it cannot carry. Below capacity both are zero, which the operating-regime discussion already anticipated.

Because a loss model is a modelling choice and not a fact, we ran three that disagree where it matters. Compounding the per-link survival along the path gives 30.9%; charging only the bottleneck link gives 36.5%; sharing each link max-min fairly among the flows crossing it gives 37.1%. The advantage is not an artifact of any one of them. We report the range rather than the most favourable member.

I. Against a cheap-but-strong baseline

Sections VI-A to VI-E compare against a naive floor and an expensive ceiling. Neither is competitive by construction. Here we place a policy in the cell between them: split each demand equally across the paths within $(1 + \varepsilon)$ of shortest on free-flow cost. It is congestion blind, costs one pass, and predates every learned method.

Textbook equal-cost multipath is a no-op on this topology. Inter-satellite delays take continuous values, so of 2620 node-destination pairs examined not one had two out-edges of exactly equal cost, and $\varepsilon = 0$ reduces to blind single-path routing. The tolerance is therefore not a detail but the whole

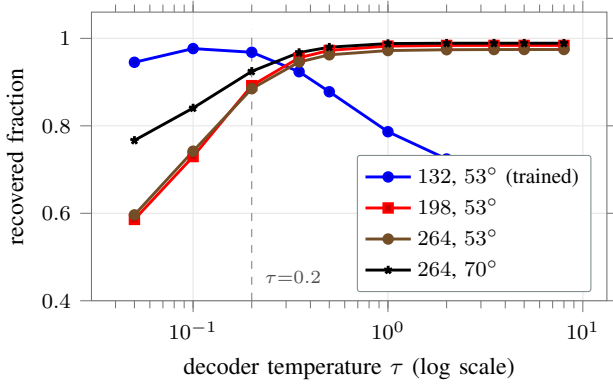


Fig. 9. The decoder temperature is not a constant of the method. On the shell the model was trained on the curve peaks near $\tau = 0.1$ and falls away; on every larger or differently inclined shell it rises and then plateaus one to two orders of magnitude higher. The fixed value inherited from the training shell (dashed) sits far down the curve everywhere else, and the loss that causes was previously charged to the learned price field.

of the baseline, and it is a knob: we sweep it rather than fix it.

Both sides are tuned per shell over fixed grids, the learned decoder over its softmin temperature and the baseline over ε , with eight instances per shell. On the shell the model was trained on it recovers 0.977 of the blind-to-equilibrium gap against 0.668; paired per instance the median difference is 0.317. On three shells it never saw, the same comparison reads 0.984 against 0.976, 0.974 against 0.973, and 0.989 against 0.969, with paired differences of 0.008, 0.002 and 0.020. The learned field is ahead on more instances than not in every case, but at this margin and this sample size the defensible reading is parity, not advantage. The advantage that is large is the in-distribution one.

The reason the baseline is strong off distribution, and weak on the training shell, is structural rather than incidental. Spreading needs somewhere to spread: on the 132-satellite grid the blind multipath split recovers 0.660 of the gap, and on a shell twice the size it recovers 0.970, because a larger grid offers more near-shortest detours between the same pair. Its strength grows precisely in the regime where a learned map is asked to generalize.

J. Two causes of the off-distribution gap, and they are separable

At the fixed decoder temperature used throughout the earlier sections, transfer looks much worse than the previous subsection reports: 0.892, 0.885 and 0.925 on the three unseen shells. The difference is not noise and it is not one effect.

The first is the training distribution. Holding the number of training instances fixed at twelve and varying only where they come from, a model shown three small shells instead of one recovers 0.976 on an unseen 264-satellite shell against 0.898 for the single-shell model. The effect is not uniform and we report all three test shells rather than the one that flatters it: on a 396-satellite shell diversity helps by a similar margin, 0.881 to 0.953, while on the 264-satellite shell at 70° inclination it hurts, 0.950 to 0.934, even though the mixed set included a

70° shell at training scale. Diversity in size transfers to size; diversity in inclination did not transfer to inclination here, and we do not know why.

The second has not been reported before as far as we are aware, and is of comparable size. The softmin temperature that decodes prices into split flows is treated in the literature, and in our own earlier sections, as a constant. It is not. Sweeping it, as Fig. 9 shows, the setting that maximises recovery moves by one to two orders of magnitude between the smallest shell and the larger ones, and holding it at the value chosen on the smallest costs a median of 0.094, 0.089 and 0.065 per instance on the three unseen shells. On the training shell the same fixed value costs 0.007, which is the point: it was chosen there, and a temperature that is nearly free where it was selected is not thereby free anywhere else. A temperature tuned on a small constellation under-spreads on a large one, and the resulting loss is charged to the price field, which did not cause it. Any evaluation of transfer that holds the decoder fixed is measuring the decoder as much as the model.

K. Reconciling the numbers that appear more than once

Several quantities in this paper were measured more than once, by experiments written at different times for different questions, and the resulting entries do not agree exactly. The clearest case is the recovered fraction of the 264-shell at the fixed decoder temperature, which appears three times: the transfer sweep gives 0.915, the tuned comparison gives 0.885, and the scaling table gives 0.93. Each traces correctly to the CSV that produced it, and the pipeline that checks every number against its source passed all three, because provenance and consistency are different properties and only the first was being checked.

Pooling all 17 units we have measured for that quantity gives a median of 0.889 over a range of 0.838 to 0.923. That range, not any single entry, is the honest summary. We report it here rather than quietly selecting one value, and we have added a check that flags any quantity carrying two entries whose difference is not accounted for by a declared run set.

The spread matters for one claim and not for the others. It does not affect the decoder-temperature result of Section VI-J, because that comparison is paired within a single instance: both arms use the same seed, the same slot and the same demand matrix, so between-run variation cannot enter it. Measured that way the cost of holding τ at 0.2 on this shell is 0.089 and every one of the eight units is on the same side, against a between-run standard deviation of 0.020. It does affect any reading that compares an entry in one table with an entry in another, which is why we now give the range wherever a quantity appears twice.

A second, smaller inconsistency belongs here. Table III reports medians over units, because the comparisons it summarises are paired and a median of paired differences is the robust statistic for them. Tables VI, VII, VIII and XI report means with standard deviations, because they describe the spread within a cell rather than compare two policies. Both choices are defensible on their own and we have kept them, but the earlier version declared “median throughout” in one place

while printing means in four others, which invited exactly the cross-table comparison that the two estimators do not support. Each table now states which estimator it uses, and where a quantity appears under both we give the pooled range above rather than inviting the reader to subtract one from the other.

L. Operating regime

The value of the method is regime dependent, and Table IV makes the boundary explicit. Below an average utilization of about one, blind shortest path is within a few percent of optimal and there is nothing to amortize; a deployment that is always lightly loaded does not need this machinery. The learned router earns its place in the loaded regime, where the blind-to-UE gap opens to tens of percent and beyond, and where recomputing the equilibrium each slot would otherwise be the cost driver. This is also the regime a capacity-planned network spends its busy hours in, and the regime a moving hotspot creates locally even when the network as a whole is moderately loaded, since the hotspot concentrates demand onto a small cut. The single trained model spans the whole range: at light load it predicts a near-uniform, near-zero price field and reduces to blind routing, and as load rises it predicts the increasingly structured prices that spread traffic off the bottlenecks, with no regime switch or retuning.

M. Feature ablation

The blind-load feature is what makes the price predictable. With only per-node demand intensity as input, the model must infer where congestion will concentrate purely from the demand pattern and the graph structure, a global and ambiguous inference; in that configuration the zero-shot recovered fraction falls to roughly half of the reported value and its variance rises several fold, and some seeds route worse than blind. Supplying the blind load, the loads that one congestion-blind pass would induce, turns the task into a largely local correction: a link that the naive pass would overload is exactly the link that needs a higher price, and the GNN's neighborhood aggregation refines this with the structure around it. The feature costs a single AON pass to compute, the same pass already used to build the node features, so it is effectively free. This also explains why the global propagation operators of Section VI-N do not help once the feature is present: the long-range information they were meant to gather is already summarized, locally, by the blind load.

N. Operator ablation, including the quantum walk (negative)

Table XI compares three param-matched propagation operators: a local graph convolution (GCN), a global classical-diffusion operator (Heat), and a global quantum-walk operator (QW). All three use the multipath decode. Once the blind-load features are present, predicting the price is largely a local function of a link's own load, so the plain GCN is best out of distribution ($0.94 \pm .01$) and the global operators add variance without benefit: the quantum walk is both the lowest and the most variable ($0.90 \pm .06$). The quantum walk originally motivated this work as a way to capture long-range structure;

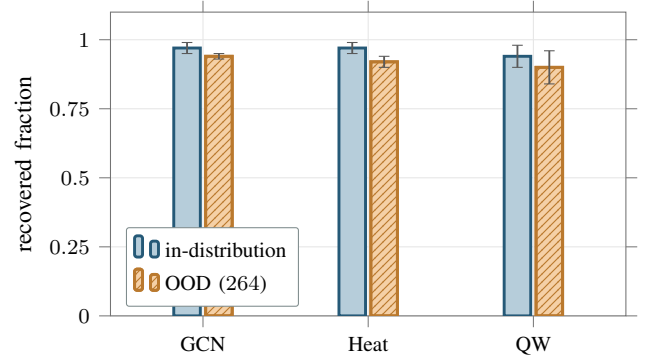


Fig. 10. Operator ablation with error bars (recovered fraction of the blind-to-UE gain, mean $\pm$ std, $n=12$). The local GCN is best out of distribution; the global quantum-walk operator does not help and is the most variable.

TABLE XI
OPERATOR ABLATION: RECOVERED FRACTION OF THE BLIND-TO-UE GAIN (MEAN $\pm$ STD, $n=12$). THE PLAIN GCN IS BEST OUT OF DISTRIBUTION; THE GLOBAL QUANTUM-WALK OPERATOR DOES NOT HELP AND IS THE MOST VARIABLE. *Run set:* THIS TABLE IS COMPUTED FROM ITS OWN INDEPENDENT SET OF SEEDS. POOLING EVERY UNIT WE HAVE MEASURED FOR THE 264-SHELL AT $\tau = 0.2$, ACROSS ALL THREE RUN SETS, GIVES A MEDIAN OF 0.889 SPANNING 0.838 TO 0.923; THE ENTRIES HERE AND IN THE OTHER TABLES ARE POINTS INSIDE THAT RANGE, NOT CORRECTIONS OF ONE ANOTHER (SECTION VI-K).

operator	in-distribution	OOD (264)
GCN (local)	0.97 $\pm$.02	0.94$\pm$.01
Heat (global diffusion)	0.97 $\pm$.02	0.92 $\pm$.02
QW (global quantum walk)	0.94 $\pm$.04	0.90 $\pm$.06

the data did not support it, and we report it as an honest negative ablation rather than force it into the contribution. Figure 10 shows the same comparison with error bars.

VII. DISCUSSION, THREATS, AND LIMITATIONS

A. Deployment considerations

In an operational control loop the router runs once per slot on the known next-slot topology. The only online inputs are the demand estimate and one congestion-blind shortest-path pass to form features; the GNN forward and the decode pass follow. Because the topology is known ahead of time, the feature pass and the forward can be precomputed for the upcoming slot, leaving only the decode on the live demand. The model is small (a few thousand parameters) and its weights are fixed across constellation sizes, so a single trained model serves an entire program of shells. Two safeguards follow naturally from our analysis. When the demand forecast is unreliable, for example during an unmodeled surge, the controller can fall back to reactive features or to blind routing, which our proactive sweep shows is preferable to acting on a badly stale estimate beyond the crossover drift. And because the decoded routing is a valid flow on a downhill DAG of positive-cost edges, it can never produce a loop or a disconnected route, unlike a directly predicted next-hop policy that must be guarded against such failures.

B. Threats to validity

We separate threats we controlled from those that remain. *Construct validity*: total travel time under the BPR cost is the standard, well-grounded traffic-assignment objective and is survivorship free by Section V; it is a flow-level model, and a packet-level study (Section VII-D) is the natural confirmation that the predicted prices route well when delay is realized by queues rather than a cost function. *Internal validity*: every learned number is param-matched across operators, averaged over multiple instances and seeds with standard deviations, and features are scale-normalized so the model cannot shortcut on absolute coordinates; the operator ablation isolates the propagation operator at a fixed parameter budget. *External validity*: results are on synthetic Walker shells with a uniform capacity and a gravity demand model, and the zero-shot transfer was tested up to a $12\times$ size jump and one shell geometry; other geometries, heterogeneous link capacities, and real traffic traces remain to be studied. *Reproducibility*: the pipeline is deterministic from parameters and every figure number is regenerated from a result CSV by a released script.

C. What the measurements do and do not license

The simulator is kinematic and flow level, not packet level; it captures geometry-driven delay and load-induced congestion through the BPR model, the mechanism the method targets, but it does not model queueing dynamics or protocol behavior, and a packet-level study (for example on Hypatia) is the natural next validation. UE is a selfish equilibrium and only a lower bound away from SO; in our regime the two nearly coincide (price of anarchy up to 1.12), so the learned target is near-optimal, but at other capacity-to-demand ratios the gap could matter and SO would be the better target. The zero-shot transfer leaves a residual gap to UE on the larger shell; training across a range of shell sizes, rather than a single small shell, is a natural way to close it. The proactive variant assumes the demand drift is predictable, which holds for the diurnal rotation of the dense-traffic band but not for sudden, unforecastable surges, where a reactive fallback would be needed. Finally, our multipath decode closes most but not all of the gap to UE; the residual decoder floor δ_{dec} (about 3.6%, Section VI-D) reflects the softmax split's deviation from the exact equilibrium splitting, which an equilibrium-faithful or learned decoder could shrink further.

D. Future work

Several directions follow directly. A packet-level study, for example on Hypatia [3], would replace the flow-level BPR proxy with queueing and protocol dynamics and test whether the predicted price field still routes well when delay is realized by buffers rather than a cost function. An equilibrium-faithful or learned decoder, going beyond the softmax downhill split to reproduce the exact UE flow splitting, is the most direct route to closing the residual decoder floor δ_{dec} . Heterogeneous and time-varying capacities, including feeder-link and ground-segment bottlenecks, would stress the assumption of a uniform ISL capacity and likely make the price field richer and more

valuable to learn. Training across a spectrum of shell sizes, rather than a single small shell, should improve the zero-shot transfer and reduce its variance. On the proactive side, replacing the known-drift forecast with a learned demand predictor would let the method handle hotspots whose motion is only statistically predictable, with the reactive and blind fallbacks of Section VII-A as a safety net when the forecast is poor. Finally, light online fine-tuning of the released weights on a target constellation could recover the few points of optimality lost in zero-shot transfer while preserving the one-shot inference cost.

VIII. CONCLUSION

We presented an inductive graph network that amortizes congestion-aware traffic engineering for LEO mega-constellations. After formalizing routing under load as the amortization of a traffic-assignment equilibrium and decomposing it into four sub-problems, we solved the price-prediction core with a GNN that predicts the user-equilibrium congestion price on every ISL in one forward pass from cheap load features. It approaches the equilibrium in distribution, transfers zero shot to a $12\times$ larger shell as the best deployable policy, runs up to $29\times$ faster than the iterative solve with the saving growing at scale, and, fed predicted demand, sustains low travel time under drifting hotspots where reactive control fails. We documented four evaluation pitfalls and a protocol to avoid them, and we reported why an initially appealing quantum-walk operator did not earn its place. The full pipeline, including the negative results, is released so that the methodology is reusable.

DATA AND CODE AVAILABILITY

The simulator, models, experiments, and figure-generation scripts are released at <https://github.com/haodpsut/qwgnn-leo-routing> (the repository name reflects the project's original quantum-walk direction, which the paper reports as a negative ablation). The constellation and traffic models are in `sim/`; the edge-price GNN and the experiments producing each claim are in `experiments/` (p4 headroom, p5 router and operator ablation, p6 baselines and timing, p7 proactive, poa_check price of anarchy); every figure number is aggregated from `results/*.csv` by `paper/make_figs_data.py`. All results are reproducible from parameters with no external data dependency.

APPENDIX A REPRODUCIBILITY NOTES

All learned results use $K=3$ layers, hidden width 32, the Adam optimizer, and are averaged over multiple demand instances and random seeds with standard deviations reported. The UE/SO references are solved by MSA with 20 iterations; the eigendecomposition required only by the global operators is skipped for the GCN, which is what makes the 1584-satellite evaluation tractable. Features are scale-normalized so the model cannot shortcut on absolute coordinates or counts.

REFERENCES

- [1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proc. ACM HotNets*, 2018, pp. 85–91.
- [2] D. Bhattacharjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proc. ACM CoNEXT*, 2019, pp. 341–354.
- [3] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the 'Internet from space' with Hypatia," in *Proc. ACM IMC*, 2020, pp. 214–229.
- [4] Z. Lai, H. Li, and J. Li, "StarPerf: Characterizing network performance for emerging mega-constellations," in *Proc. IEEE ICNP*, 2020, pp. 1–11.
- [5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.
- [6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.
- [7] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.
- [8] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, 2020.
- [9] P. Almasan, J. Suárez-Varela, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio, "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *Comput. Commun.*, vol. 196, pp. 184–194, 2022.
- [10] J. G. Wardrop, "Some theoretical aspects of road traffic research," *Proc. Inst. Civil Eng., Part II*, vol. 1, no. 3, pp. 325–362, 1952.
- [11] Bureau of Public Roads, *Traffic Assignment Manual*. Washington, DC, USA: U.S. Dept. of Commerce, 1964.
- [12] M. Beckmann, C. B. McGuire, and C. B. Winsten, *Studies in the Economics of Transportation*. New Haven, CT, USA: Yale Univ. Press, 1956.
- [13] T. Roughgarden and É. Tardos, "How bad is selfish routing?" *J. ACM*, vol. 49, no. 2, pp. 236–259, 2002.
- [14] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. ACM HotNets*, 2017, pp. 185–191.
- [15] Z. Xu *et al.*, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE INFOCOM*, 2018, pp. 1871–1879.
- [16] Z. Gu, J. Choi, T. Q. S. Quek, and J. Park, "Duality-guided graph learning for real-time joint connectivity and routing in LEO mega-constellations," arXiv:2601.21921v1 [eess.SP], Jan. 2026. Submitted to an IEEE journal for possible publication.
- [17] C. Hopps, "Analysis of an equal-cost multi-path algorithm," RFC 2992, Internet Engineering Task Force, Nov. 2000.
- [18] S. Dernbach, A. Mohseni-Kabir, S. Pal, and D. Towsley, "Quantum walk neural networks for graph-structured data," in *Complex Networks and Their Applications VII* (Stud. Comput. Intell., vol. 813). Cham, Switzerland: Springer, 2018, pp. 182–193.
- [19] O. Kotheli *et al.*, "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, 2021.
- [20] M. Giordani and M. Zorzi, "Non-terrestrial networks in the 6G era: Challenges and opportunities," *IEEE Network*, vol. 35, no. 2, pp. 244–251, 2021.
- [21] I. del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three low Earth orbit satellite constellation systems to provide global broadband," *Acta Astronaut.*, vol. 159, pp. 123–135, 2019.
- [22] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at Starlink performance," in *Proc. ACM IMC*, 2022, pp. 130–136.
- [23] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. ICML*, 2017, pp. 1263–1272.
- [24] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. ICLR*, 2019.
- [25] P. W. Battaglia *et al.*, "Relational inductive biases, deep learning, and graph networks," arXiv:1806.01261, 2018.
- [26] J. Suárez-Varela *et al.*, "Graph neural networks for communication networks: Context, use cases and opportunities," *IEEE Network*, vol. 37, no. 3, pp. 146–153, 2023.
- [27] F. Geyer and G. Carle, "Learning and generating distributed routing protocols using graph-based deep learning," in *Proc. ACM SIGCOMM Big-DAMA Workshop*, 2018, pp. 40–45.
- [28] O. Hope and E. Yoneki, "GDDR: GNN-based data-driven routing," in *Proc. IEEE ICDCS*, 2021, pp. 517–527.
- [29] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1985.
- [30] M. Frank and P. Wolfe, "An algorithm for quadratic programming," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 95–110, 1956.
- [31] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proc. IEEE INFOCOM*, 2000, pp. 519–528.
- [32] S. Kandula, D. Katabi, B. Davie, and A. Charny, "Walking the tightrope: Responsive yet stable traffic engineering," in *Proc. ACM SIGCOMM*, 2005, pp. 253–264.
- [33] C.-Y. Hong *et al.*, "Achieving high utilization with software-driven WAN," in *Proc. ACM SIGCOMM*, 2013, pp. 15–26.
- [34] S. Jain *et al.*, "B4: Experience with a globally-deployed software defined WAN," in *Proc. ACM SIGCOMM*, 2013, pp. 3–14.
- [35] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. ACM HotNets*, 2016, pp. 50–56.
- [36] L. Chen, J. Lingys, K. Chen, and F. Liu, "AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization," in *Proc. ACM SIGCOMM*, 2018, pp. 191–205.
- [37] Z. Liu *et al.*, "KAN: Kolmogorov-Arnold networks," arXiv:2404.19756, 2024.
- [38] B. Amos, "Tutorial on amortized optimization," *Foundations and Trends in Machine Learning*, vol. 16, no. 5, pp. 592–732, 2023.



Phuc Hao Do is a Lecturer and Researcher in the Faculty of Information Technology at Danang Architecture University (DAU), Viet Nam. He received the Ph.D. degree in telecommunications systems and networks from the Bonch-Bruевич Saint Petersburg State University of Telecommunications (SPbSUT), Russia, in 2026. His research interests include intelligent 6G non-terrestrial networks, quantum-inspired AI, Kolmogorov-Arnold networks (KAN), resilient computing, and network security. He has authored peer-reviewed papers in international journals and conferences on IoT malware detection, satellite communication optimization, and deep learning architectures.



Nang Hung Van Nguyen received his Ph.D. degree in Computer Science from The University of Danang, Vietnam, in 2021. He is currently a lecturer at The University of Danang - University of Science and Technology. His research interests include Artificial Intelligence, Machine Learning, Geometric Algebra, and Computer Networking.



Minh Tuan Pham received his Ph.D. degree in Computational Science and Engineering from Nagoya University, Japan, in 2012. He is currently a lecturer in the Faculty of Information Technology, The University of Danang – University of Science and Technology, Danang, Vietnam. His research interests include Artificial Intelligence, Soft Computation, and Geometric Algebra.